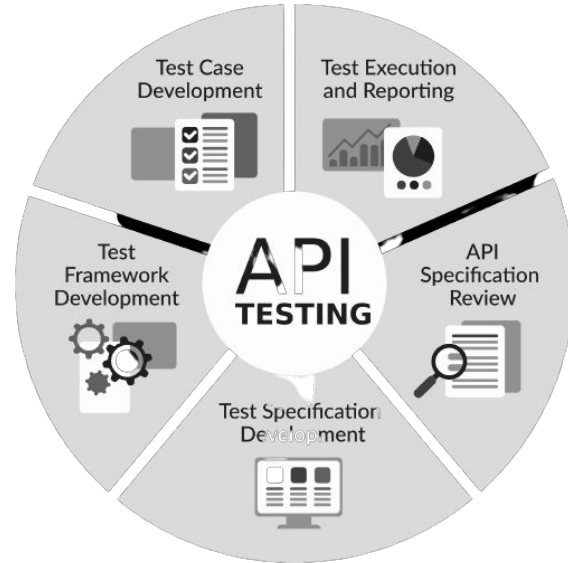


TestNG Mastery



Pramod Dutta
Lead SDET.



TestNG Mastery



TestNG

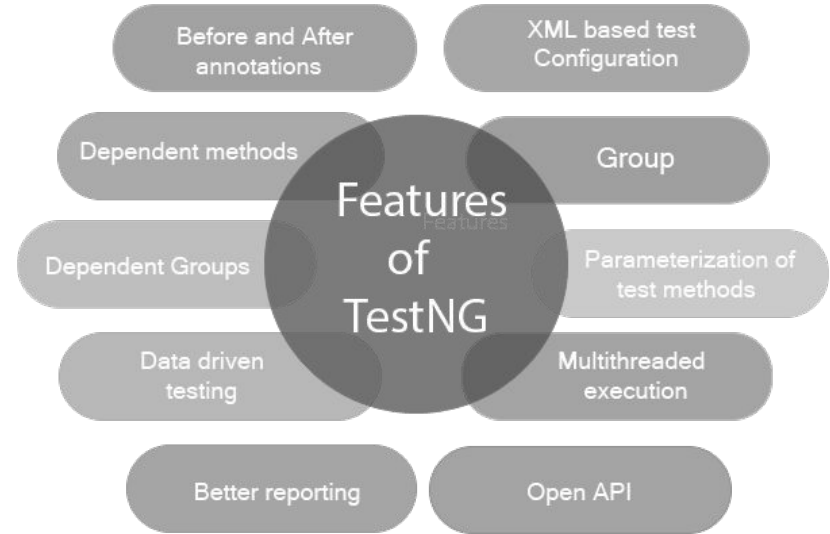
401
Response

What is TestNG?

- Easy to use **testing framework** written in Java.
- Designed for *unit*, **functional**, and end-to-end integration tests.
- You can use it with the **API Testing** and **UI Testing**.
- Provides features like parallel testing, multi thread execution and annotations support.
- **TestNG supports multiple plugin** and can be integrated easily with existing frameworks.

Why we Need TestNG?

1. Reporting
2. Parallel Execution
3. Annotations Support
4. Multi threaded testing
5. Easy integration
6. Open source
7. Better than JUnit Framework



How to Write Test in TestNG?

1. Create a Method and add your logic in that Method
2. Mark that Method as @Test
3. Add required Annotations like
 - a. description,
 - b. group,
 - c. priority,
 - d. enabled,

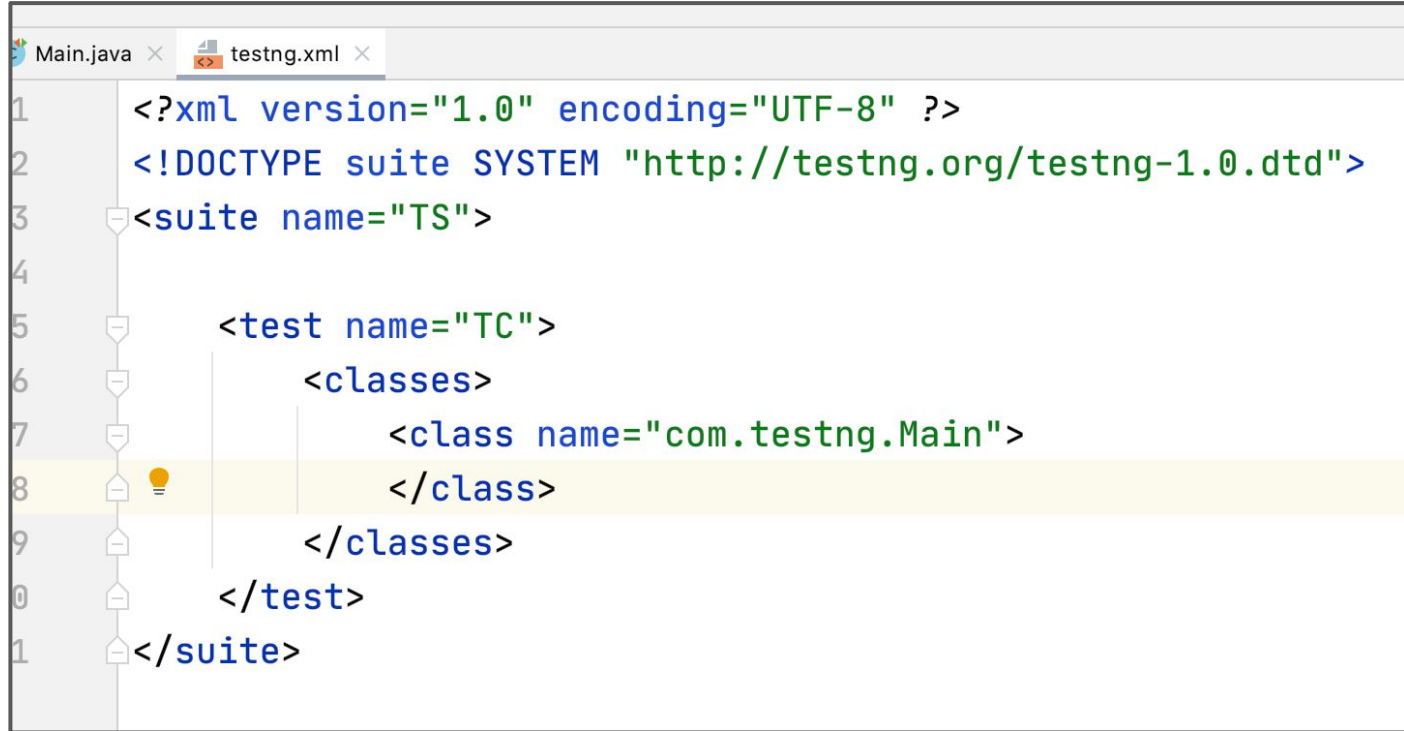
How to Write Test?

```
1 package com.testng;  
2  
3 import org.openqa.selenium.WebDriver;  
4 import org.openqa.selenium.firefox.FirefoxDriver;  
5 import org.testng.Assert;  
6 import org.testng.annotations.Test;  
7  
8 public class Main {  
9     @Test  
10    void openGoogle(){  
11        WebDriver driver = new FirefoxDriver();  
12        driver.get("https://scrolltest.com");  
13        Assert.assertEquals(driver.getTitle(), expected: "Scrolltest - Software Testing & Automation");  
14        driver.quit();  
15    }  
16 }  
17
```

What is TestNG File?

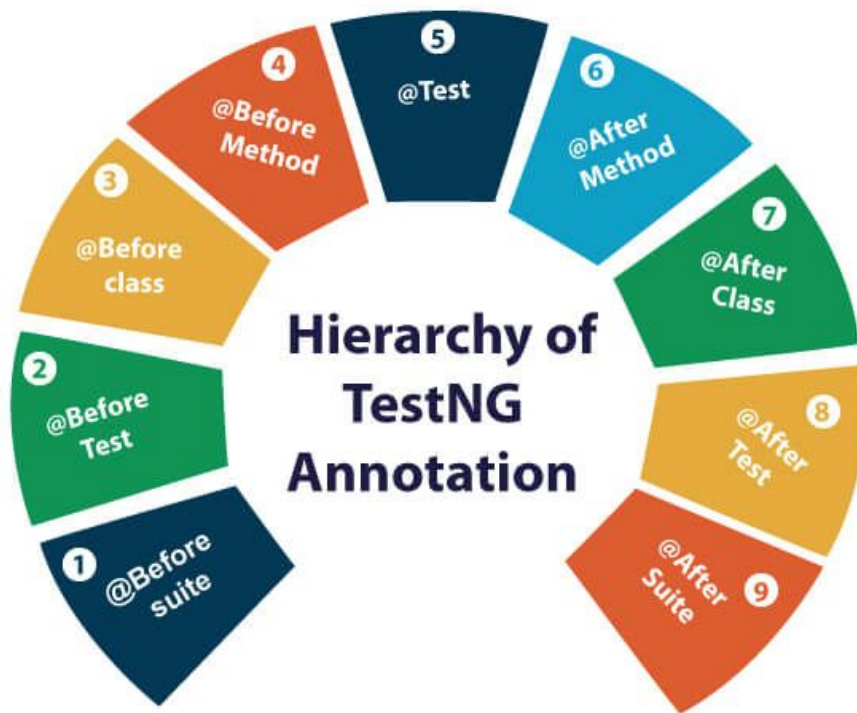
1. It's a XML format file.
2. It's a file that contains the test configuration.
3. It allows us to organize test classes.
4. You can define test suites and tests.
5. You can add parameters and other parallel config support here.

What is TestNG File?



```
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3  <suite name="TS">
4
5      <test name="TC">
6          <classes>
7              <class name="com.testng.Main">
8                  </class>
9          </classes>
10     </test>
11 </suite>
```


TestNG Annotations



TestNG Annotations

@BeforeSuite: The annotated method will be run before all tests in this suite have run.

@AfterSuite: The annotated method will be run after all tests in this suite have run.

@BeforeTest: The annotated method will be run before any test method belonging to the classes inside the <test> tag is run.

@AfterTest: The annotated method will be run after all the test methods belonging to the classes inside the <test> tag have run.

@BeforeGroups: The list of groups that this configuration method will run before. This method is guaranteed to run shortly before the first test method that belongs to any of these groups is invoked.

@AfterGroups: The list of groups that this configuration method will run after. This method is guaranteed to run shortly after the last test method that belongs to any of these groups is invoked.

TestNG Annotations

@BeforeClass: The annotated method will be run before the first test method in the current class is invoked.

@AfterClass: The annotated method will be run after all the test methods in the current class have been run.

@BeforeMethod: The annotated method will be run before each test method.

@AfterMethod: The annotated method will be run after each test method.

TestNG Annotations

@Parameter annotation on test method is to pass parameters to test methods.

@DataProvider annotated method is used to create test methods or test classes at runtime with different parameters.

@Factory can be used on a method that returns instances of test classes or on a test class constructor in conjunction with @DataProvider

TestNG Annotations

@Listeners is used at test class level to takes the array of classes that implements a plethora of implementations of ITestNGListener interface like IAlterSuiteListener, IAnnotationTransformer, IMethodInterceptor, IReporter, etc. for different purposes.

Grouping of Tests.

What is the grouping of tests?

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="test_suite">
  <groups>
    <run>
      <include name="SmokeTest"/>
    </run>
  </groups>
  <test name="Personal Loan">
    <classes>
      <class name="com.javatpoint.Personal_loan"/>
    </classes>
  </test> <!-- Test -->
```

```
package com.javatpoint;
import org.testng.annotations.Test;
public class Personal_loan
{
    @Test(groups= {"SmokeTest"})
    public void WebLoginPersonalLoan()
    {
        System.out.println("Web Login Personal Loan");
    }
    @Test
    public void MobileLoginPersonalLoan()
    {
        System.out.println("Mobile Login Personal Loan");
    }
    @Test
    public void APILoginPersonalLoan()
    {
        System.out.println("API Login Personal Loan");
    }
}
```

Running Tests in Parallel

Parallelism and thread count can be set at **suite level** or **test level** like below.

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">  
<suite name="TS" parallel="tests" thread-count="5">
```

Running Tests in Parallel

Methods run test methods in parallel in different threads. All dependent methods will be run in different threads, respecting the priority of tests.

Tests run `<test>` tags in parallel in separate threads.

Classes run test classes in parallel in separate threads, but test methods in those test classes will run in the same thread.

Instances run instances of test methods/classes in parallel in different threads.

Running Tests in Parallel

@DataProvider, the parallelism can be controlled using the attribute @DataProvider(**parallel = true**)



Listeners

Listeners are TestNG annotations that **literally “listen” to the events in a script** and modify TestNG behaviour accordingly

- IAnnotationTransformer
- IExecutionListener**
- IHookable
- IInvokedMethodListener
- IMethodInterceptor
- IReporter
- ISuiteListener
- ITestListener**



ReportNG

Easy to use Reporting

```
<dependency>  
  <groupId>org.testng</groupId>  
  <artifactId>reportng</artifactId>  
  <version>1.2.2</version>  
  <scope>test</scope>  
</dependency>
```



Listeners

Demo of IExecutionListener

```
CustomListener.java
1 package com.thetestingacademy.testng.Listener;
2
3 import org.testng.IExecutionListener;
4
5 public class CustomListener implements IExecutionListener {
6     @Override
7     public void onExecutionFinish() {
8         long endTime= System.currentTimeMillis();
9         System.out.println("****   *** Finished execution at- " + endTime + "
10
11     }
12
13     @Override
14     public void onExecutionStart() {
15         long startTime= System.currentTimeMillis();
16         System.out.println(" ****   *** Started execution at - " + startTime
17
18     }
19 }
20
```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3 <suite name="All Test Suite">
4     <listeners>
5         <listener class-name="com.thetestingacademy.testng.Listener.CustomLis
6     </listeners>
7     <test name="LearnTestNG">
8
9         <groups>
10             <run>
11                 <include name = "sanity"></include>
12                 <exclude name="smoke"></exclude>-->
13             </run>
14         </groups>
15
16         <classes>
17             <class name="com.thetestingacademy.testng.Groups"></class>
18         </classes>
19
20     </test>
21 </suite>
```



Data Provider CSV File

```
<dependency>
```

```
<groupId>com.codoid.products</groupId>
```

```
<artifactId>fillo</artifactId>  
  <version>1.15</version>  
</dependency>
```

User, pass x4

Passed as array to function()

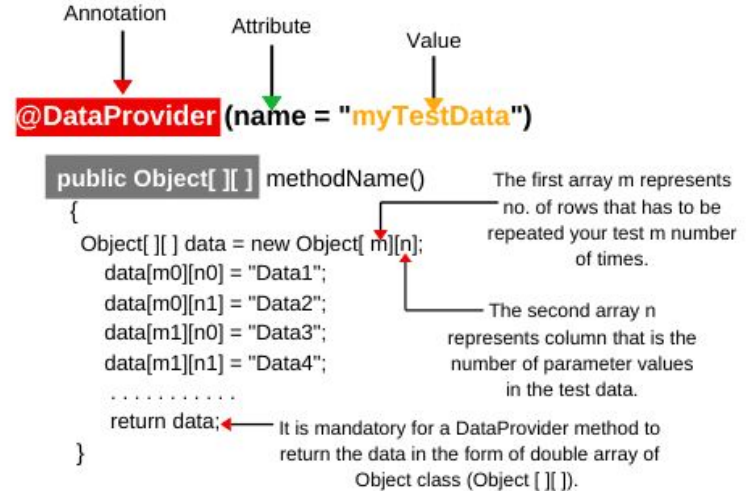


Fig: TestNG DataProvider Annotation | A complete code structure



Data Provider CSV File

```
Run All
11 public class BasicTest2 {
12
13     @Test(dataProvider = "dp1")
        Run | Debug
14     public void TestLogin(String[] s) throws Exception {
15         System.out.println(s[0] + " >> [" + s[1]);
16     }
17
18     @DataProvider()
19     public String[][] dp1() {
20         String[][] data= new String[][] {
21             {"hyr", "123"},
22             {"pqr", "456"},
23             {"xyz", "789"}
24         };
25         return data;
26     }
27 }
```



TestNG – Parallel Test Execution

TestNG parallel execution of tests, classes and suites with examples. Learn how to run testng tests and suites in parallel or single test in multiple threads.

Test Level

Reduces execution time

Class Level

Allows multi-threaded tests

Method Level

Data Provider

Thread-Count

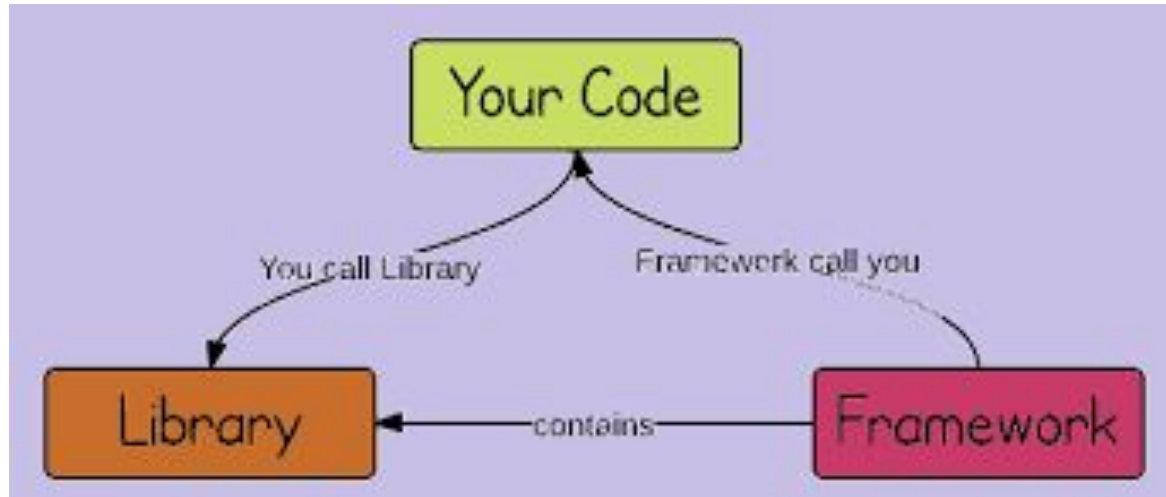


Why Framework



Why Framework

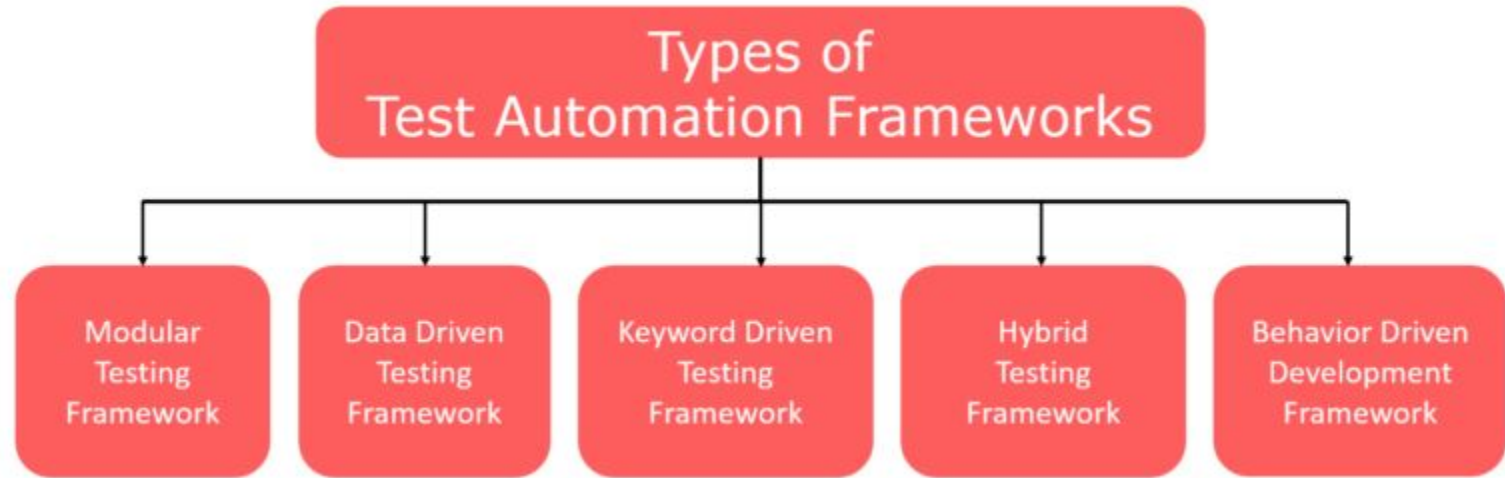
A framework defines the organization's way of doing things - a 'Single Standard'

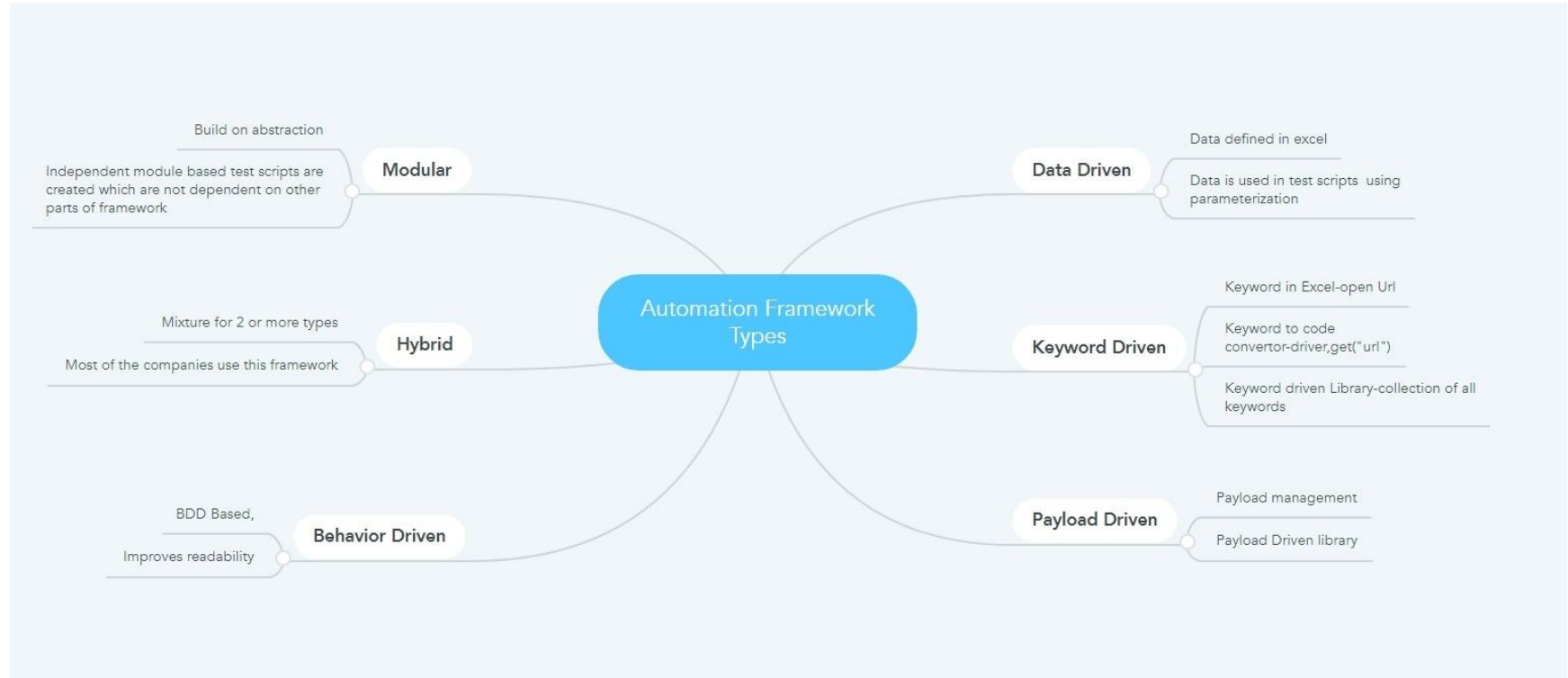




Framework vs Library

Basis	LIBRARY	FRAMEWORK
Meaning	Library is collection of reusable functions used by computer program	Framework is a piece of code that dictates the architecture of project
Inversion of control	With a library , you are in-charge, means you can choose where and when you want to insert or use the library.	In a framework, the framework is in-charge, not you, means a framework tells you where to put a specific part of your code
Function	They are important in program linking and binding process	In a framework , provided standard way to build and deploy applications
Flexibility	Libraries are more flexible with greater degree of control	Frameworks are enforced structure and standards.
Example	React.js, JQuery is a javascript library	Angular js, Vue js is javascript framework







Modular

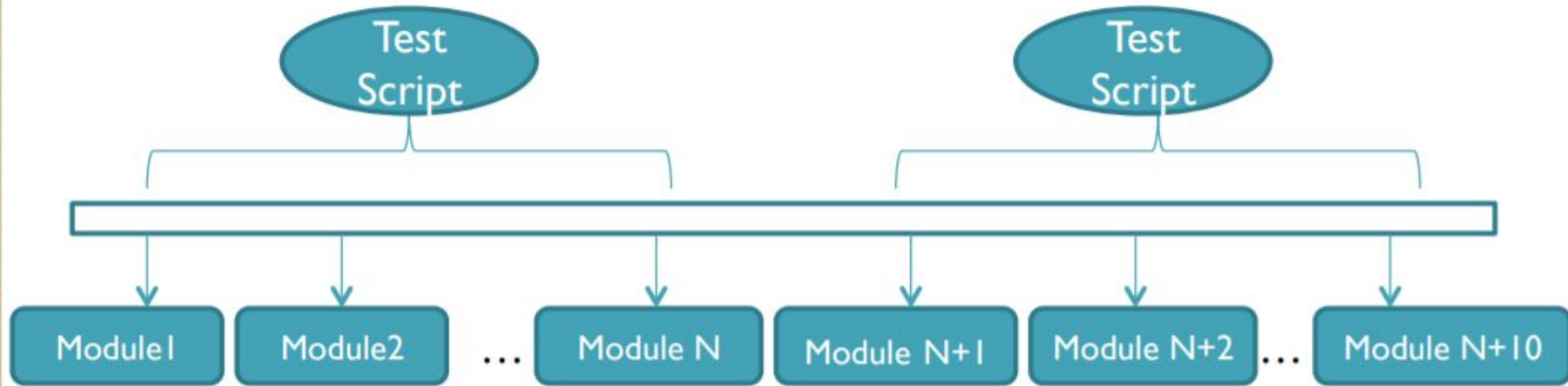
Modular frameworks divides the test scripts into small modules where modules are small scripts written to perform certain tasks.

Modular framework is like creation of small, independent scripts that represents modules, sections and functions of the application under test

individual test scripts can be combined to make larger test scripts by using a master script to achieve the required scenarios.

Master script is used to invoke the individual modules to run end to end test scenarios

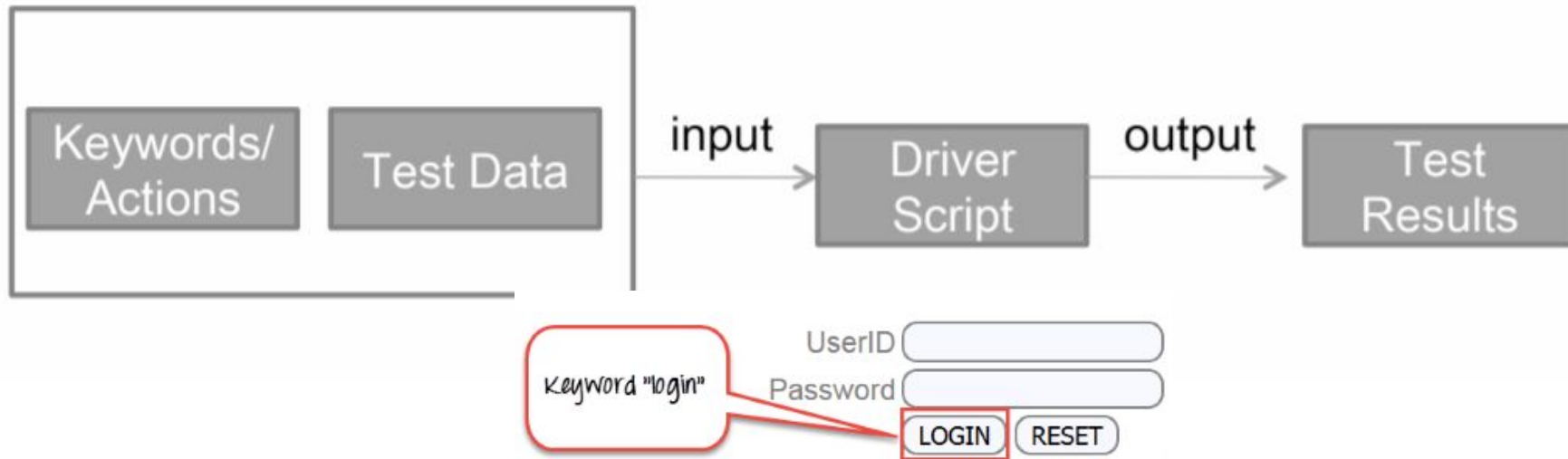
Modular



Keyword Driven



Keyword Driven Framework is a functional automation testing framework that divides test cases into four different parts in order to separate coding from test cases and test steps for better automation



Keyword Driven

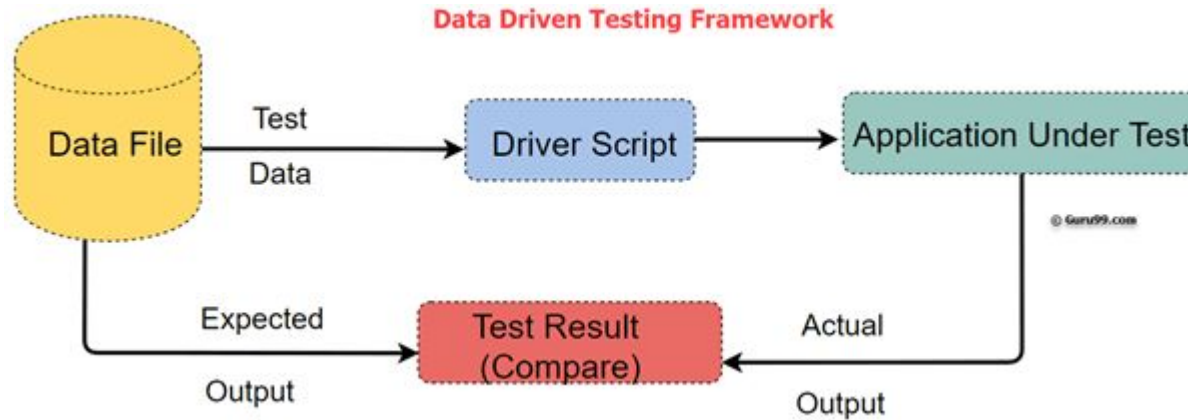


ID	Name	Step_ID	Description	Keyword	
	1 Login to Application	Step 1	Open browser	openBrowser	
		Step 2	Navigate to URL	navigate	
		Step 3	Enter Email	enterEmail	
		Step 4	Enter Password	enterPassword	
		Step 5	Click on Sign in button	clickSignIn	
		Step 6	Click on Logout button	logout	
		Step 7	Close browser	closeBrowser	

Data Driven

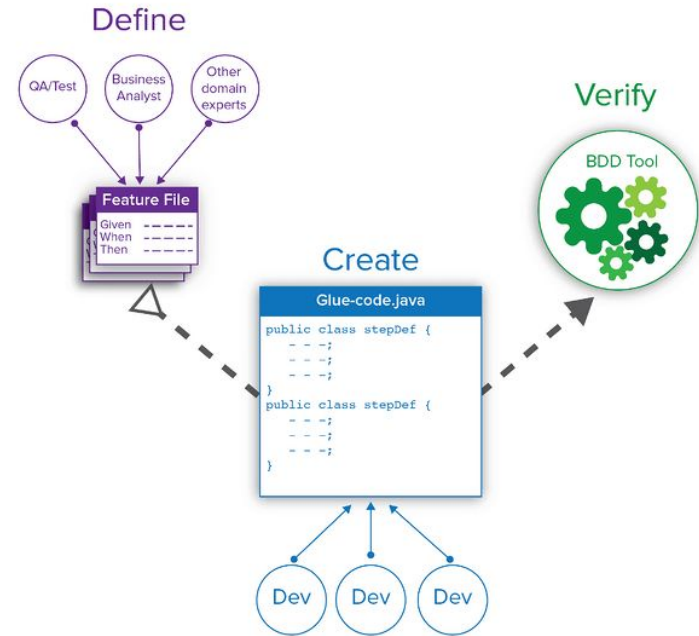
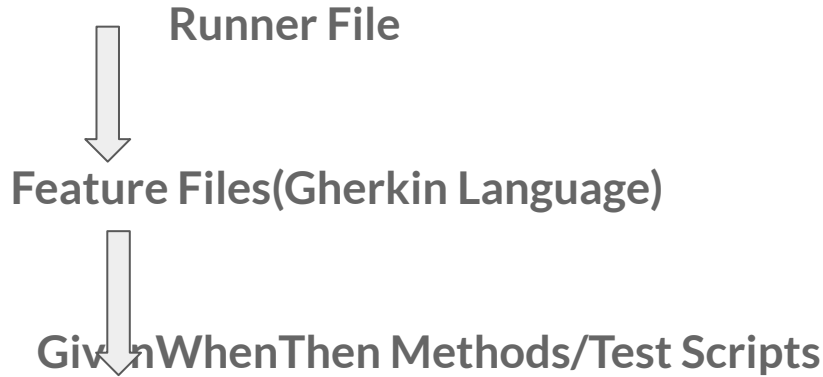


Data driven testing allows testers to input a single test script that can execute tests for all test data from a table and expect the test output in the same table





Behavior Driven

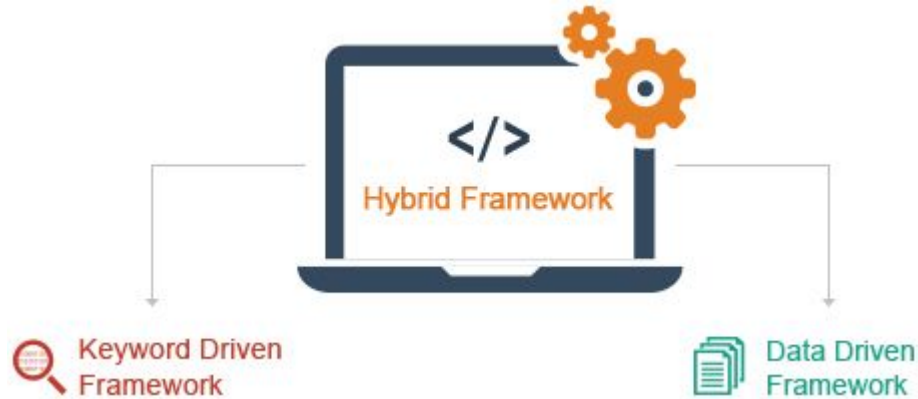


Hybrid



Combination of multiple types

=>Data Driven+BDD+Keyword





Understand Requirements

- Is Framework Needed?
- Select Programming Language and tool
- Choose Your Framework Design.
- Create your framework
- Implement CI/CD

Framework Components



- Manage Dependencies/Projects(Maven/Gradle/PIP/NPM/Nuget)
- Manage Data(Excel/json files/prop files/xml files)
- Manage Payload and Endpoints(json strings/jsonmaps/pojos/serialize/deserialize/gson)
- Manage Tests (testng and allure for this)(precondition/postcondition/set/config/teardown/steps,description,priority,severity/execution)
- Reuse Components(Keywords, Abstraction, Inheritance, Generics, Configs. Specs. setups/helpers)
- Logger-Report Loggers(Testng. allure), text loggers(log4J)
- Reports-Test summary, percentage, steps, description, failure reason, logs-allure
- Utils -String manipulators, Json Manipulators, Data Manipulators, Readers/Writers/custom code/tools
- CI/CT-Version Controlling-github/git, Continuous Integration and Testing-jenkins/teamcity/travisci

API Framework Creation



1. Folder Structure
2. Payload in the class format
3. Utils of the Excel and Fillow into the framework
4. Allure report to the framework
5. Git
6. Run via Jenkins in the local machine

Thanks, for attending Class

I hope you liked it.

Fin.